

University of Engineering and Technology

Name:

Fatima Saeed

Registration Number:

2026(s)-CYS-15(A)

Program:

BS Cyber Security

Course:

Programming Fundamentals (PF)

Submitted To:

Sir Hafiz Muhammad Mubashar



University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamental	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: First	Session: Spring 2026
Lab/Project/Assignment #: 1	CLOs to be covered: CLO1
Lab Title: Input/Output, Data Types, Operators, Conditional Statements(if/else)	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 1

Lab Goals/Objectives:

To learn about Data Types, Operators, Conditional Statements.

Equipment Required:

Computer System with VS code installed on it.

Theory:

- **Input/Output:** Input is used to take data from the user, and output is used to display the result on the screen.
- **Data Types:** Data types define the kind of data a variable can store. Common data types are int (integer), float (decimal number), char (single character), and string (text).
- **Operators:** Operators perform different operations on data. Arithmetic operators (+, -, *, /) are used for calculations, while comparison operators (>, <, ==) compare values.
- **Conditional Statements (if/else):** These statements help the program make decisions. If a condition is true, the if block is executed; otherwise, the else block is executed.

Lab Work:

Task 1:

```
#percentage

a = int(input("enter obtained marks: "))
b = int(input("enter total marks: "))

c = a/b*100
print(c)
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft VS Code>
1 percentage.py"
enter obtained marks: 33
enter total marks: 40
82.5
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft VS Code>
```

Task 2:

```
# basic calculations

a = 38
b = 4
c = a % b
print(c)
d = a / b
print(d)
e = c + d
print(e)
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft VS Code>
4 calculations.py"
2
9.5
11.5
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft VS Code>
```

Task 3:

```
obtained_marks = int(input("Enter your marks: "))
total_marks = int(input("Enter total marks: "))

if total_marks <= 0 or total_marks > 300:
    print("Enter valid numbers")
elif obtained_marks > total_marks:
    print('Enter valid numbers')
else:
    result = (obtained_marks/total_marks) * 100
    print(result)
if result >= 90:
    print("A+")
elif result >= 85:
    print('A-')
elif result >= 80:
    print('B+')
elif result >= 75:
    print('B-')
elif result >= 70:
    print("C+")
elif result >= 65:
    print("C-")
elif result >= 60:
    print('F')
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft VS Code>
2 grades.py"
Enter your marks: 30
Enter total marks: 40
75.0
B-
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft VS Code>
```

Conclusions:

In this lab, I learned the basics of python programming. I understood how to take input from the user, display output, use different data types and operators, and apply **if/else** statements to make decisions. This lab improved my understanding of basic programming concepts and prepared me for more advanced programming topics.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamental	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: First	Session: Spring 2026
Lab/Project/Assignment #: 2	CLOs to be covered: CLO1
Lab Title: For Loops	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 2

Lab Goals/Objectives:

To learn about For Loops.

Equipment Required:

Computer System with VS code installed on it.

Theory:

A **for loop** in Python is used to repeat a block of code a specific number of times. It makes programs shorter and avoids writing the same code again and again. In Python, the `range()` function is commonly used with a for loop to generate a sequence of numbers. They make programs more efficient and easier to understand.

Lab Work:

Task 1:

```
Students = int(input("Enter total number of students:"))
for i in range(Students):
    Student_name = input("Emter student name:")
    Roll_Number = int(input("Enter roll no:"))
    Obtained_marks = int(input("enter your marks:"))
    Total_Marks = int(input("enter total marks:"))
    print("Student name:", Student_name)
    print("Roll number:", Roll_Number)
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Program
oop.py"
Enter total number of students:10
Emter student name:Fatima
Enter roll no:15
enter your marks:20
enter total marks:30
Student name: Fatima
Roll number: 15
66.66666666666666
C-
```

Conclusions:

In this lab, I learned how to use **for loops** in Python to execute a set of statements repeatedly. I understood the use of the `range()` function and how loops can simplify programming tasks by reducing repetitive code. This lab improved my understanding of loops and helped me write basic Python programs more efficiently.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: First	Session: Spring 2026
Lab/Project/Assignment #: 3	CLOs to be covered: CLO1
Lab Title: While Loops	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 3

Lab Goals/Objectives:

To learn about While Loops.

Equipment Required:

Computer System with VS code installed on it.

Theory:

A **while loop** in Python is used to repeat a block of code as long as a given condition is **True**. Before each iteration, the condition is checked. If the condition is true, the loop continues, otherwise, it stops. A while loop is useful when the number of repetitions is not known in advance. It is important to update the loop variable inside the loop to avoid an infinite loop.

Lab Work:

Task 1:

```
Students = int(input("Enter total number of students:"))
i = 1
while i <= (Students):
    Student_name = input("Emter student name:")
    Roll_Number = int(input("Enter roll no:"))
    Obtained_marks = int(input("enter your marks:"))
    Total_Marks = int(input("enter total marks:"))
    print("Student name:", Student_name)
    print("Roll number:", Roll_Number)
    result = (Obtained_marks/Total_Marks) * 100
    if Total_Marks <= 0 or Total_Marks > 300:
        print("enter valid numbers:")
    elif Obtained_marks > Total_Marks > 300:
        print("Enter valid numbers:")
    else:
        print(result)
        if result >= 90:
            print("A+")
        elif result >= 85:
            print("A-")
        elif result >= 80:
            print("B+")
        elif result >= 75:
            print("B-")
        elif result >= 70:
            print("C+")
        elif result >= 65:
            print("C-")
    i = i + 1
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft  
loop.py"  
Enter total number of students:30  
Enter student name:Fatima  
Enter roll no:15  
enter your marks:33  
enter total marks:50  
Student name: Fatima  
Roll number: 15  
66.0  
C-
```

Conclusions:

In this lab, I learned how to use **while loops** in Python to repeat statements based on a condition. I understood how the loop starts, continues while the condition is true, and stops when the condition becomes false. This lab improved my understanding of repetition in programming and helped me write simple Python programs using while loops.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: First	Session: Spring 2026
Lab/Project/Assignment #: 4	CLOs to be covered: CLO1
Lab Title: Nested Loops	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 4

Lab Goals/Objectives:

To learn about Nested Loops.

Equipment Required:

Computer System with VS code installed on it.

Theory:

A **nested loop** is a loop inside another loop. In Python, both **for loops** and **while loops** can be nested. The inner loop executes completely every time the outer loop runs once. Nested loops are commonly used to print patterns, work with tables, and process two-dimensional data such as matrices. They help solve problems that require repeating one task inside another.

Lab Work:

Task 1:

```
sets > FILES > Documents > week 1 > Nested.py
for i in range(1,10):
    for j in range(i):
        print("*",end = "")
    print()
```

Result:

```
Nested.py
*
**
***
****
*****
*****
*****
*****
*****
*****
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft VS Code>
```

Task 2:

```
for i in range(1,3):
    for j in range(1,3):
        print('i=', i , 'j=' ,j)
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft V
1.py"
i= 1 j= 1
i= 1 j= 2
i= 2 j= 1
i= 2 j= 2
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft V
```

Task 3:

```
for i in range(1,5):
    for j in range(i + 1):
        print(j,end = '')
    print()
print("\n","\n")
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft
2.py"
01
012
0123
01234
```

Task 4:

```
for i in range(1,5):
    for j in range(1,5):
        print(i*j,'\t',end = ' ')
    print()
print("\n","\n")
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft Windows Common-  
4.py"  
1      2      3      4  
2      4      6      8  
3      6      9      12  
4      8      12     16
```

Task 5:

```
for i in range(1,5):  
    for j in range(4-i):  
        print(' ', end = ' ')  
    for j in range(2*i-1):  
        print('*', end = ' ')  
    print()  
print("\n", "\n")
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft Windows Common-  
7.py"  
 *  
 ***  
 *****  
 *****
```

Conclusions:

In this lab, I learned how to use **nested loops** in Python. I understood that a loop can be placed inside another loop and that the inner loop runs for every iteration of the outer loop. I also learned how nested

loops are used to print patterns and perform repetitive tasks efficiently. This lab improved my understanding of loops.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: First	Session: Spring 2026
Lab/Project/Assignment #: 5	CLOs to be covered: CLO2
Lab Title: Functions	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 5

Lab Goals/Objectives:

To learn about Functions.

Equipment Required:

Computer System with VS code installed on it.

Theory:

A **function** in Python is a block of code that performs a specific task. Functions help make programs more organized, reusable, and easier to understand. A function is created using the `def` keyword, and it can be called whenever needed. Functions can take **parameters** (inputs) and may **return** a value using the `return` statement. By using functions, programmers can avoid repeating the same code multiple times and make programs easier to maintain and debug.

Lab Work:

Task 1:

```
Users > PMLS > Downloads > week 11 > ta  
def hello():  
    print("Hello World")  
  
hello()
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Python\Python39-64\Scripts> python 1.py  
Hello World  
PS C:\Users\PMLS\AppData\Local\Programs\Python\Python39-64\Scripts>
```

Task 2:

```
Users > PMLS > Downloads > week 11 > ta  
def sum(a,b):  
    return a+b  
  
c = sum(2,5)  
print(c)
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Microsoft\Windows\PowerShell\PowerShellProfiles\1\AllProfiles\Task3\Task3.ps1
3.py"
7
PS C:\Users\PMLS\AppData\Local\Microsoft\Windows\PowerShell\PowerShellProfiles\1\AllProfiles\Task3\Task3.ps1
```

Task 3:

```
e = max(90,100)
print(e)

b = len("Fatima")
print(b)

f = min(-20,30)
print(f)
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Microsoft\Windows\PowerShell\PowerShellProfiles\1\AllProfiles\Task3\Task3.ps1
5.py"
100
6
-20
```

Task 4:

```
def info(name,roll):  
    print(f'name: {name} \nRoll_no: {roll}')  
    print(f'My name is {name}')
```



```
info('Fatima',15)
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft VS Code>  
7.py"  
name: Fatima  
Roll_no: 15  
My name is Fatima
```

Conclusions:

In this lab, I learned how to create and use **functions** in Python. I understood how to define a function, pass arguments to it, and return values. I also learned that functions make programs more organized, reduce code repetition, and improve readability. This lab strengthened my understanding of modular programming and helped me write Python programs more efficiently.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: First	Session: Spring 2026
Lab/Project/Assignment #: 6	CLOs to be covered: CLO2
Lab Title: Global and Local Variables	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 6

Lab Goals/Objectives:

To learn about Local and Global Variables.

Equipment Required:

Computer System with VS code installed on it.

Theory:

Variables are used to store data in a Python program. A **local variable** is created inside a function and can only be used within that function. It is automatically removed when the function finishes execution. A **global variable** is declared outside all functions and can be accessed from anywhere in the program. If a global variable needs to be modified inside a function, the `global` keyword is used.

Lab Work:

Task 1:

```
a= 25
def x():
    global a
    a= 50
    print(f"Value of a inside function", a)

print(a)
x()
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft VS Code
  3 global.py"
25
Value of a inside function 50
```

Task 2:

```
x = '15'  
y = 'Fatima'  
print(x + y)  
def a():  
    print(x)  
  
a()
```

Result:

```
PS C:\Users\PMLS\AppData\Loc  
al v.py"  
15Fatima  
15
```

Conclusions:

In this lab, I learned the difference between **local** and **global variables** in Python. I understood that local variables are limited to the function in which they are created, while global variables can be accessed throughout the program. I also learned how to use the `global` keyword when modifying a global variable inside a function. This lab improved my understanding of variable scope and helped me write better-structured Python programs.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: First	Session: Spring 2026
Lab/Project/Assignment #: 7	CLOs to be covered: CLO3
Lab Title: List, Tuple	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 7

Lab Goals/Objectives:

To learn about List, Tuple.

Equipment Required:

Computer System with VS code installed on it.

Theory:

A **list** in Python is an ordered and **mutable** (changeable) collection of items. Lists are created using square brackets `[]` and can store different types of data such as numbers, strings, and Boolean values. Items in a list can be added, removed, or modified after the list is created.

A **tuple** is also an ordered collection of items, but it is **immutable** (cannot be changed after creation). Tuples are created using parentheses `()`. They are useful for storing data that should remain constant throughout the program. Both lists and tuples support indexing and slicing, making it easy to access their elements.

Lab Work:

Task 1:

```
#LIST

a= [1,2,3,4,5,6]
print(a[0])
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Pro
3 list.py"
1
```

Task 2:

```
fruits = ("apple", "banana", "mango")
print(fruits)

print(fruits[0])
print(fruits[2])

student = ("Fatima", 20, "A+")
print(student)
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft  
10.py"  
( 'apple', 'banana', 'mango' )  
apple  
mango  
( 'Fatima', 20, 'A+' )
```

Conclusions:

In this lab, I learned about **lists** and **tuples** in Python. I understood that lists are mutable and can be modified, while tuples are immutable and cannot be changed after they are created. I also learned how to create, access, and use both data structures in Python programs. This lab improved my understanding of storing and managing collections of data efficiently.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: First	Session: Spring 2026
Lab/Project/Assignment #: 8	CLOs to be covered: CLO3
Lab Title: Sets, Dictionary	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 8

Lab Goals/Objectives:

To learn about Sets, Dictionary.

Equipment Required:

Computer System with VS code installed on it.

Theory:

A **set** in Python is an unordered collection of **unique** elements. Sets are created using curly braces `{}` or the `set()` function. They do not allow duplicate values and are useful for storing distinct items. Python provides operations such as **union**, **intersection**, and **difference** to perform mathematical set operations.

A **dictionary** is a collection of **key-value pairs**. It is also created using curly braces `{}`, where each key is associated with a value. Dictionary keys must be unique, while values can be repeated. Dictionaries are used to store and retrieve data efficiently using keys instead of indexes. They support operations such as adding, updating, and deleting key-value pairs.

Lab Work:

Task 1:

```
Users > PMLS > Documents > week 7 > dictionary.py > fruits  
fruits = {"Apple": " seeb", " gava": " amrood", "mallberry": "shatoot", "mango":"aam"}  
print(fruits)
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft VS Code> & C:\Users\PMLS\AppData\Local\Programs\Microsoft VS Code> python dictionary.py  
{'Apple': ' seeb', ' gava': ' amrood', 'mallberry': 'shatoot', 'mango': 'aam'}
```

Task 2:

```

b = {'CE', 'CS', 'CE', 'EE'}
b.add('AU')
print(b)

b.add('CE')
print(b)

b.discard('CS')
print(b)

b.discard('CE')
print(b)

b.discard('VE')
print(b)

b.remove('VE')
print(b)

```

Result:

```

PS C:\Users\PMLS\AppData\Local\Prog
s.py"
{'EE', 'CE', 'AU', 'CS'}
{'EE', 'CE', 'AU', 'CS'}
{'EE', 'CE', 'AU', 'CS'}
{'EE', 'AU', 'CS'}
{'EE', 'AU', 'CS'}

```

Task 3:

```
b = {'CE', 'CS', 'CE', 'EE'}
for i in b:
    print(i)

if 'CE' in b:
    print('yes')
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Python\Python37\python.exe:
7.py"
CS
CE
EE
yes
```

Task 4:

```
s1 = {2,4,5}
s2 = {'a','b',2}
s = s1.symmetric_difference(s2)
print(s)
print(s1)
print(s2)

s1 = {2,4,5}
s2 = {'a','b',2}
s3 = s1.difference(s2)
print(s3)
```

Result:

```
PS C:\Users\PMLS\AppData\Local\Programs\Microsoft  
5.py"  
{4, 5, 'a', 'b'}  
{2, 4, 5}  
{'a', 2, 'b'}  
{4, 5}
```

Conclusions:

In this lab, I learned about **sets** and **dictionaries** in Python. I understood that sets store only unique values and are useful for performing set operations, while dictionaries store data in the form of key-value pairs for fast and efficient access. I also learned how to create, access, modify, and use both data structures in Python programs. This lab improved my understanding of Python collections and their practical applications in programming.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated: July 1st, 2026
Semester: 1st	Session: Spring 2026
Lab/Project/Assignment #: 10	CLOs to be covered: CLO
Lab Title: Recursion	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 10

Lab Goals/Objectives:

- To understand the concept of recursion in Python
- To learn how to define a recursive function with a base case and recursive case
- To apply recursion to solve mathematical problems like factorial and Fibonacci

- To understand how recursive function calls work on the call stack

Equipment Required:

Computer system with VS code and Python package installed on it.

Theory:

Recursion is a technique where a function calls itself to solve a problem by breaking it into smaller subproblems. Every recursive function must have a base case (a condition that stops the recursion) and a recursive case (where the function calls itself with a simpler input). Without a base case, the function will call itself infinitely and cause a RecursionError. Recursion is commonly used to solve problems like factorial, Fibonacci series, and countdown timers.

Task 1:

Write a recursive program that counts down from a given number to 1 and prints each value.

```
1
2  #code
3  def show(n):
4      if n == 0:
5          return
6      if n > 0:
7          print(n)
8          show(n-1)
9  show(5)
10
```

Result:

The program defines a recursive function show() that prints the current value of n and calls itself with n-1 until it reaches the base case (n == 0). When called with 5, it prints numbers 5 down to 1.

```
PS C:\Users\ayfak> & C:/Users/ayfak/Python/Scripts/python.exe C:/Users/ayfak/Python/Scripts/python.exe
5
4
3
2
1
PS C:\Users\ayfak>
```

Task 2:

Write a recursive program that calculates the factorial of a number.

```
1
2  #code
3  def factorial(n):
4      if n == 0 or n == 1:
5          return 1
6      else:
7          return n * factorial(n-1)
8  print("The factorial is : ", factorial(5))
9
```

Result:

The program defines a recursive factorial() function where the base case returns 1 when n is 0 or 1, and the recursive case multiplies n by the factorial of n-1. When called with 5, it calculates $5 \times 4 \times 3 \times 2 \times 1$ and prints 120.

```
PS C:\Users\ayfak> & C:/Users/ayfak/AppData
The factorial is : 120
PS C:\Users\ayfak>
```

Task 3:

Write a recursive program that calculates the nth Fibonacci number.

```
1
2  #code
3  def fibonacci(n):
4      if n <= 0:
5          return 0
6      elif n == 1:
7          return 1
8      else:
9          return fibonacci(n-1) + fibonacci(n-2)
10 print("The fibonacci is :", fibonacci(3))
```

Result:

The program defines a recursive fibonacci() function where the base cases return 0 for $n \leq 0$ and 1 for $n=1$. For any other value, it recursively adds the previous two Fibonacci numbers. When called with 3, it returns 2.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\ayfak> & C:/Users/ayfak/AppData/Local/P
The fibonacci is : 2
PS C:\Users\ayfak>
```

Conclusions:

This lab helped me understand how recursion works in Python. By implementing recursive programs for countdown, factorial, and Fibonacci, I learned how every recursive function needs a base case to stop the recursion and a recursive case that reduces the problem step by step until the base case is reached.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated: July 1st, 2026
Semester: 1st	Session: Spring 2026
Lab/Project/Assignment #: 11	CLOs to be covered: CLO
Lab Title: Exception Handling	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 11

Lab Goals/Objectives:

- To understand the concept of exceptions and errors in Python
- To learn how to use try and except blocks to handle exceptions
- To understand the difference between specific and general exception handling
- To learn how to write robust programs that do not crash on invalid input

Equipment Required:

Computer system with VS code and Python package installed on it.

Theory:

An exception is an error that occurs during the execution of a program and disrupts the normal flow of instructions. Python provides the try/except block to handle exceptions gracefully without crashing the program. Code that might raise an error is placed inside the try block, and the corresponding handler is written in the except block. Specific exceptions like ValueError and ZeroDivisionError can be caught individually, or a general Exception can be used to catch any error. Code outside the try/except block continues to execute regardless of whether an exception occurred.

Task 1:

Write a program that handles invalid input using a try/except block with ValueError.

```
1
2  #code
3  try:
4      a = int(input("Enter a number: "))
5      print(a)
6  except ValueError:
7      print("Invalid input.")
8  print("Hello World")
9
```

Result:

The program uses a try/except block to handle a ValueError. If the user enters a valid integer it prints the number. If the user enters something invalid like a string, it catches the error and prints Invalid input. Either way Hello World is printed at the end since it is outside the try/except block.

```
PS C:\Users\ayfak> & C:/Users
Enter a number: papa
Invalid input.
Hello World
PS C:\Users\ayfak>
```

Task 2:

Write a program that handles any type of exception using a general except block.

```
1
2  #code
3  try:
4      a = int(input("Enter a number: "))
5      print(a)
6  except Exception as e:
7      print("An error occured:", e)
8  print("Hello World")
9
```

Result:

The program uses a try/except block with a general Exception handler that catches any type of error and prints the error message using e. If valid input is given the number is printed. Either way Hello World prints at the end since it is outside the try/except block.

```
PS C:\Users\ayfak> & C:/Users/ayfak/AppData/Local/Python/pythoncore-  
Enter a number: 24  
24  
Hello World  
PS C:\Users\ayfak> & C:/Users/ayfak/AppData/Local/Python/pythoncore-  
Enter a number: aoa  
An error occured: invalid literal for int() with base 10: 'aoa'  
Hello World  
PS C:\Users\ayfak> █
```

Conclusions:

This lab helped me understand how exception handling works in Python. By using try/except blocks, I learned how to prevent programs from crashing on invalid input, how to catch specific errors like ValueError, and how to use the general Exception class to handle any unexpected error gracefully.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated: July 1st, 2026
Semester: 1st	Session: Spring 2026
Lab/Project/Assignment #: 12	CLOs to be covered: CLO
Lab Title: Lambda Functions	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 12

Lab Goals/Objectives:

- To understand the concept of lambda (anonymous) functions in Python
- To learn how to define and use lambda functions for simple operations
- To understand how to use lambda functions with map(), filter(), and reduce()
- To compare lambda functions with regular user-defined functions

Equipment Required:

Computer system with VS code and Python package installed on it.

Theory:

A lambda function in Python is a small, anonymous function defined in a single line using the lambda keyword. Unlike regular functions defined with def, lambda functions do not have a name and can only contain a single expression. The syntax is: lambda parameters: expression. Lambda functions are commonly used with higher-order functions like map(), filter(), and reduce() which apply a function to elements of a list or sequence.

Task 1:

Write a program that defines and calls a function to calculate and print the square of a number.

```
1
2  #code
3  def square(n):
4      print(n**2)
5      return n**2
6  square(5)
7
```

Result:

The program defines a square() function that takes a number, prints its square using **, and also returns the value. When called with 5, it prints 25.

```
PS C:\Users\ayfak> & C:\
25
PS C:\Users\ayfak> █
```

Task 2:

Write a program that uses a lambda function to calculate the square of a number.

```
#code
a = lambda x: x*x
```

Result:

The program defines a lambda function that takes one parameter x and returns $x*x$. When called with 5, it prints 25, demonstrating how lambda functions work as short one-line alternatives to regular functions.

```
PS C:\Users\ayfak> & C:/L  
81  
PS C:\Users\ayfak> █
```

Task 3:

Write a program that uses a lambda function to add three numbers together.

```
#code  
a = lambda x,y,z: x+y+z  
print(a(7,11,12))
```

Result:

The program defines a lambda function with three parameters that returns their sum. When called with 7, 11, and 12, it prints 30.

```
PS C:\Users\ayfak> & C:/Us  
30  
PS C:\Users\ayfak> █
```

Task 4:

Write a program that uses `map()` with a lambda function to square all elements in a list.

```
#code
a = [1, 2, 3, 4]
s = list(map(lambda x: x*x, a))
print(s)
```

Result:

The program uses `map()` with a lambda function to apply the squaring operation to each element of the list `a`. The result is converted back to a list using `list()` and printed, showing `[1, 4, 9, 16]`.

```
PS C:\Users\ayfak> & C:/Us
[1, 4, 9, 16]
PS C:\Users\ayfak> 
) 0 Δ 0
```

Conclusions:

This lab helped me understand lambda functions in Python and how they provide a concise way to write simple functions in a single line. By implementing programs using lambda with `map()`, I learned how lambda functions are especially useful when combined with higher-order functions to process lists efficiently.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated: July 1st, 2026
Semester: 1st	Session: Spring 2026
Lab/Project/Assignment #: 13	CLOs to be covered: CLO
Lab Title: MAP, Filter and Reduce	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 13

Lab Goals/Objectives:

- To understand the concept of higher-order functions in Python
- To learn how to use map() to apply a function to every element of a list
- To learn how to use filter() to extract elements that satisfy a condition
- To learn how to use reduce() to combine elements of a list into a single value

Equipment Required:

Computer system with VS code and Python package installed on it.

Theory:

Python provides three built-in higher-order functions: `map()`, `filter()`, and `reduce()`. The `map()` function applies a given function to every element of an iterable and returns a map object. The `filter()` function filters elements of an iterable based on a function that returns `True` or `False`. The `reduce()` function, available in the `functools` module, applies a function cumulatively to the elements of a sequence, reducing it to a single value. These functions are often used with lambda functions for concise, functional-style programming.

Task 1:

Write a program that uses `map()` with a lambda function to double all elements in a list.

```
1
2  #code
3  numbers = [1, 2, 3, 4, 5]
4  doubled = list(map(lambda x: x * 2, numbers))
5  print("Doubled:", doubled)
6
```

Result:

The program uses `map()` with a lambda function to multiply each element in the list by 2. The result is converted to a list and printed, showing `[2, 4, 6, 8, 10]`.

```
PS C:\Users\ayfak> & C:/Users/ayfak/At
Doubled: [2, 4, 6, 8, 10]
PS C:\Users\ayfak>
```


The program uses `reduce()` from the `functools` module with a `lambda` function to cumulatively multiply all elements in the list together ($1 \times 2 \times 3 \times 4 \times 5$), resulting in 120.

```
PS C:\Users\ayfak> & C:/Users/ayfak/Python/Scripts/lambda_reduce.py
Product: 120
PS C:\Users\ayfak>
```

Conclusions:

This lab helped me understand the three key higher-order functions in Python: `map()`, `filter()`, and `reduce()`. I learned how `map()` transforms every element, `filter()` selects elements based on a condition, and `reduce()` combines all elements into a single result. These functions make code more concise and readable when combined with `lambda` functions.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated: July 1st, 2026
Semester: 1st	Session: Spring 2026
Lab/Project/Assignment #: 14	CLOs to be covered: CLO
Lab Title: OS Library, if <code>__name__ == "__main__"</code>, is vs <code>==</code>	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 14

Lab Goals/Objectives:

- To understand how to use the OS library to interact with the file system
- To learn the use of `__name__ == '__main__'` to control script execution
- To understand global and local variables and how to access them
- To learn how to read from and write to files using file handling

Equipment Required:

Computer system with VS code and Python package installed on it.

Theory:

The `os` module in Python provides functions to interact with the operating system, such as creating directories with `os.mkdir()`. The special variable `__name__` is set to `'__main__'` when a script is run directly, and to the module name when it is imported — this allows code to be conditionally executed. File handling in Python uses `open()` with modes `'r'` (read), `'w'` (write/overwrite), and `'a'` (append). Global variables are accessible inside functions, and the `globals()` function returns a dictionary of all global variables.

Task 1:

Write a program that demonstrates the use of `__name__ == '__main__'` to control which functions run when a script is executed directly.

```
1
2 #code
3 def welcome():
4     print("Welcome to the class")
5 if __name__ == "__main__":
6     welcome()
7 def goodbye():
8     print("Goodbye to you")
9 goodbye()
10
```

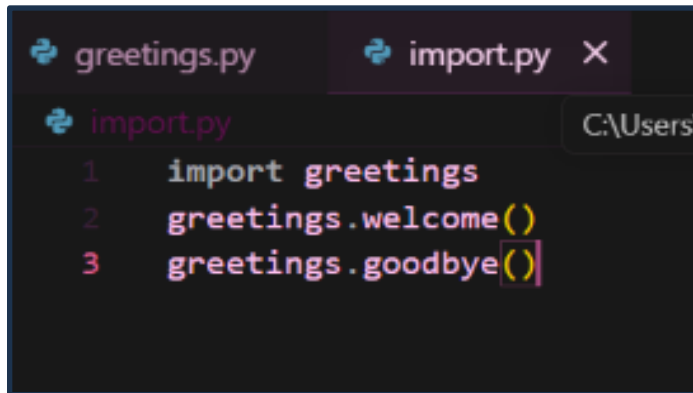
Result:

The program defines two functions, `welcome()` and `goodbye()`. The `welcome()` function is only called when the script is run directly, controlled by the `if __name__ == '__main__'` check. Both messages are printed when the script is run directly.

```
PS C:\Users\ayfak> & C:/Users/ayf
Welcome to the class
Goodbye to you
PS C:\Users\ayfak>
```

Task 2:

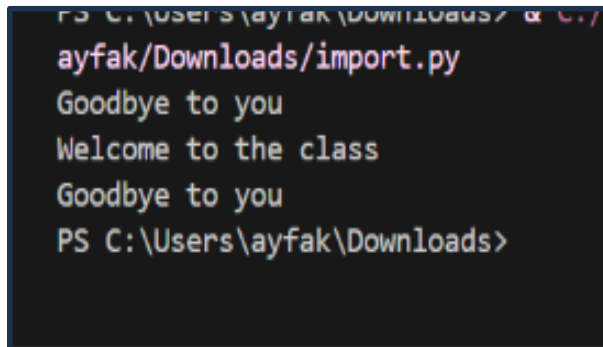
Write a program that imports a custom module and calls its functions.



```
greetings.py  import.py X
import.py
1  import greetings
2  greetings.welcome()
3  greetings.goodbye()
```

Result:

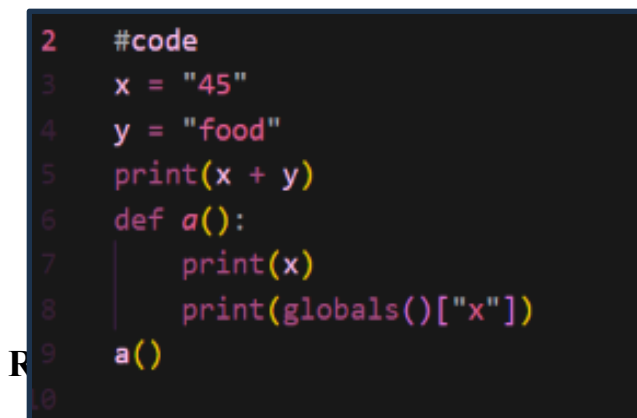
The program imports the custom module fizza (which contains welcome() and goodbye() functions defined in fizza.py) and calls both functions, printing Welcome! and Goodbye! on the screen, demonstrating how Python modules can be imported and used.



```
PS C:\Users\ayfak\Downloads> python C:/
ayfak/Downloads/import.py
Goodbye to you
Welcome to the class
Goodbye to you
PS C:\Users\ayfak\Downloads>
```

Task 3:

Write a program that demonstrates accessing a global variable inside a function using both direct access and globals().



```
2  #code
3  x = "45"
4  y = "food"
5  print(x + y)
6  def a():
7      print(x)
8      print(globals()["x"])
9  a()
10
```

R

The program accesses the global variable x inside a function both directly and using `globals()['x']`. Both methods print 45, demonstrating two ways to access global variables from within a function.

```
PS C:\Users\ayfak\Downloads> &
ayfak/Downloads/import.py
45food
45
45
PS C:\Users\ayfak\Downloads> &
ayfak/Downloads/import.py
```

Task 4:

Write a program that reads and prints the contents of the file.

```
task 4 read file(filehandling).py > ...
1
2 #code
3 c= open("DATA og file.TXT")
4 print(c.read())
5 c.close()
6
```

Result:

The program opens the file in read mode and prints the content present in it.

```
PS C:\Users\ayfak\OneDrive\Desktop\week 15> &
ayfak/OneDrive/Desktop/week 15/task 4 read f
Hi, girls
PS C:\Users\ayfak\OneDrive\Desktop\week 15>
```

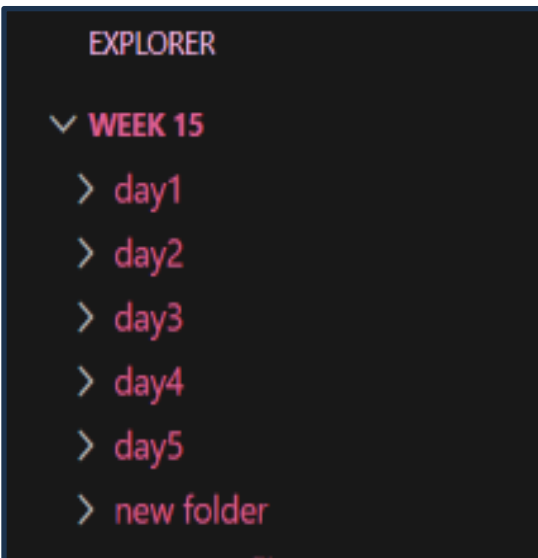
Task 5:

Write a program that creates a new directory and multiple directories using the os module.

```
#code
import os
os.mkdir("new folder")
for i in range (0, 5):
    os.mkdir(f"day{i+1}")
```

Result:

The program uses os.mkdir() to first create a single directory called new folder, then uses a for loop to create 5 directories named day_0 through day_4, demonstrating how Python can automate file system tasks using the os module.



Conclusions:

This lab helped me understand the os module, file handling, global variables, and the `__name__ == '__main__'` concept in Python. I learned how to create directories, read and write files, import custom modules, and control script execution, which are all essential skills for building real-world Python applications.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated: July 1st, 2026
Semester: 1st	Session: Spring 2026
Lab/Project/Assignment #: 15	CLOs to be covered: CLO
Lab Title: GUI using QT designer	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 15

Lab Goals/Objectives:

- To understand the basic interface and tools of Qt Designer.
- To learn how to create graphical user interfaces (GUIs) without writing code.
- To design windows using widgets such as buttons, labels, line edits, and combo boxes.
- To understand how Qt Designer .ui files are connected with Python programs.
- To develop simple, user-friendly desktop applications using PyQt5 and Qt Designer.

Equipment Required:

A computer or laptop with Python 3.x, PySide6, Qt Designer, and a code editor such as Visual Studio Code or PyCharm installed.

Theory:

Qt Designer is a graphical user interface (GUI) design tool used with the PySide6 framework. It allows developers to create application windows by dragging and dropping widgets instead of writing the entire interface in code. Widgets such as labels, buttons, line edits, combo boxes, and spin boxes can be placed easily on the window. The designed interface is saved as a **.ui** file, which can later be loaded into a Python program using PySide6. Qt Designer makes GUI development faster, easier, and more organized. It is commonly used for developing desktop applications with a user-friendly interface.

Lab work:

The screenshot shows a desktop application window for a cosmetic billing system. At the top, there is a pink button labeled 'Add to Cart', a quantity input field with the value '0', and a dropdown menu showing 'Lipstick'. Below this is a table with four columns: 'Product', 'Qty', 'Price', and 'Total'. The table is currently empty. At the bottom of the window, there are four labels and two buttons. The labels are 'Subtotal: Rs. 0.00', 'Grand Total: Rs. 0.00', 'Generate Bill' (in a green button), and 'Clear Cart' (in a red button).

This cosmetic billing system is a desktop application built using PySide6 and Qt Designer that functions like an online shopping cart. The program uses a class called `BillingWindow` which contains all the main functions. The `add_to_cart()` function is triggered when the user clicks 'Add to Cart' button. Reads the selected product and quantity, looks up the price in the dictionary, calculates the line total, inserts a new row in the cart table, and calls `update_totals()`. The `update_totals()` function loops through every row in the cart table, sums all line totals into a subtotal, computes 13% tax, calculates grand total, and updates the `subtotalLabel` and `totalLabel` widgets. The `generate_bill()` function builds the full receipt as a formatted string using a list of text lines. Iterates the cart table to add itemised rows, appends subtotal/tax/grand total, displays the bill in `QPlainTextEdit`, and saves it as a `.txt` file.

The `clear_cart()` function resets the cart by calling `setRowCount(0)` on the `QTableWidget` (removes all rows), then calls `update_totals()` to reset both labels back to Rs. 0.00. All buttons are connected to their respective functions, making the application interactive.

Conclusion:

In this lab, we learned how to use Qt Designer to create graphical user interfaces for desktop applications. We explored different widgets and learned how to arrange them properly on a window. We also understood how .ui files are connected with Python programs using PyQt5. This lab improved our understanding of GUI design and made application development easier and more efficient.
